GP2: YOLOv8s vs RT-DETR on Team 1's drives

Shanthosh Raaj Mohanram Mageswari
ENPM818Z Fall 2026, Team 1
Repository: `https://github.com/shanthoshraaj1A/GP---Team-1-ENPM818Z-Fall-2026`, branch `gp2`, tag `gp2-final`

Answers in short.

- **Budget.** YOLOv8s needs 8.8 ms per image, which is 18% of the 50 ms we have per frame and leaves 41 ms. RT-DETR-L needs 34.5 ms, or 69%, which leaves only 15.5 ms. The tracker itself takes just 0.05 ms per frame.
- **Night.** RT-DETR-L gets worse at night and YOLOv8s does not. Its mAP@0.5 drops from 0.821 to 0.760, and the biggest AP drop is for vehicles (−0.076). Counting objects one by one, pedestrians suffer most: 40 that it found by day are missed at night, and only 4 go the other way. YOLOv8s stays about the same (mAP@0.5 +0.004).
- **Tracking.** With RT-DETR-L the tracker switches IDs far more often: 43 times against 6 by day, and 28 against 9 at night. The reason is duplicate boxes. RT-DETR-L often puts two boxes on the same object with different labels, such as car and truck, and since both count as *vehicle* in GP2, the tracker sees two vehicles. On the clear drive it gave 76 such pairs, against 8 for YOLOv8s. When I merged the duplicates before tracking, its ID switches fell from 43 to 5.
- **Recommendation.** Use RT-DETR-L by day, with the duplicate boxes merged: it gains 0.078 mAP@0.5 for 25.7 ms more per frame. Use YOLOv8s at night: there RT-DETR-L is only 0.013 mAP@0.5 better, it finds pedestrians less well (AP@0.5 0.787 against 0.820), and it takes 4 times as long.

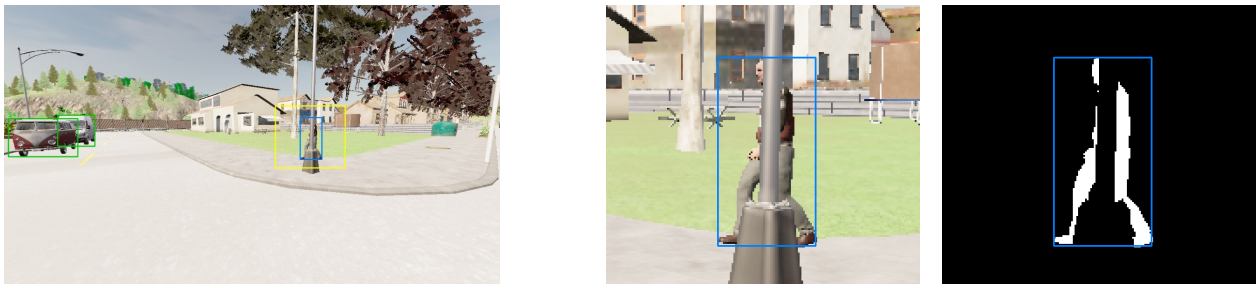
1 Setup

Both drives are Team 1's GP1 rig in CARLA Town01: a 1280×720 front camera with a 110° horizontal field of view, 600 frames at 20 Hz (30 s) each. The clear drive is ClearNoon and the night drive is ClearNight. The two drives follow the same route with the same camera poses and the same traffic, so frame i of one drive shows the same scene as frame i of the other; only the light changes. Each drive has 985 vehicle and 325 pedestrian ground-truth boxes. Both models are the COCO-pretrained weights (`yolov8s.pt`, `rtdetr-l.pt`), run at 640 px, with person mapped to *pedestrian* and car, truck and bus mapped to *vehicle*. Nothing was trained.

2 Ground truth (Task 1)

All 13 ground-truth tests pass, and the overlays for frames 50, 300 and 550 of both drives are in `results/gp2/overlays/`. Figure 1 shows the one frame where I think my ground truth is open to argument.

Why it is open to argument. The lamp post hides the middle of the pedestrian, so the camera sees two thin strips, one on each side of the pole. My ground truth draws one box around all of the visible pixels, so the box (764 to 820 px wide, 290 to 398 px tall) also covers the pole. Only 1,153 of its 6,213 pixels, 19%, are the pedestrian. That is well above the 64-pixel minimum, so the frame counts this person as a visible object that a detector should find. Neither model does: the best RT-DETR-L box on it scores 0.05 and YOLOv8s has nothing above 0.01. So both models are charged with a missed pedestrian in AP and in MOTA. One can argue either way. The person is really there and steps out two frames later (2,735 pixels in frame 72), so a safety-minded ground truth should keep them. But no detector can be expected to find a person who is 81% hidden, and the box itself is mostly pole. A fairer rule would mark boxes where under



(a) clear frame 70, ground truth **(b)** zoom (yellow area) **(c)** pedestrian pixels

Figure 1: Pedestrian 324 walks behind a lamp post. The box (blue) is drawn around the pixels the semantic and depth images say belong to the pedestrian, shown white in (c).

about 25% of the area is the actor as "ignore": neither a hit nor a miss. The same thing happens for cars behind the hedge (clear frame 300, far left).

3 Detector timing and the node (Task 2)

All 3 detector tests pass. I timed both models with `tools/run_detector.py` on an NVIDIA RTX A4500 Laptop GPU (16 GB), plugged in, in the course Docker container (ROS 2 Humble, Ultralytics 8.4.184, PyTorch 2.5.1). CARLA was closed and `nvidia-smi` showed no other process on the GPU. The time is for one `predict` call with the image already in memory, and the number is the median over the 600 frames of a drive after 2 warm-up calls. I ran every model and drive 4 times. Each run was steady (10th to 90th percentile within about 1 ms), but on a laptop GPU the clock state changes between runs, so twice a whole run came out faster (4.5 ms and 21.9 ms on the night drive). Table 1 gives the run whose detections I graded, which is the typical value.

Model	clear (ms)	night (ms)	share of 50 ms	left of 50 ms	4 runs (ms), clear / night	live node rate
YOLOv8s	8.8	8.6	18%	41.2 ms	8.8–8.9 / 4.5–8.7	20.0 Hz
RT-DETR-L	34.5	34.7	69%	15.3–15.5 ms	34.5–34.9 / 21.9–34.7	20.8 Hz

Table 1: Median time per image. Live node: night drive replayed at 20 Hz, `ros2 topic hz /perception/detections` for about 10 s (`results/gp2/node_hz.txt`).

The node runs both models from the launch file (`model_type:=yolo|detr`) and keeps up with the 20 Hz camera with either one. The saved frames are `node_yolo.png` and `node_detr.png`.

4 Detector grading (Task 3)

All 11 evaluation tests pass. AP is computed on every detection down to a score of 0.001, with my own matching, precision-recall curve and 101-point AP. `check_coco.py` agrees with pycocotools to four decimals for both models on both drives (difference +0.0000 in all eight numbers, `results/gp2/coco_check.txt`).

5 Tracking (Task 4)

All 10 tracker tests pass. The tracker is SORT-style: a constant-velocity Kalman filter per box, Hungarian matching on $1 - \text{IoU}$ per class with a gate at IoU 0.3, a track confirmed after 3 matched frames and deleted

Drive, model	AP@0.5 vehicle	AP@0.5 pedestrian	mAP@0.5	mAP@0.5:0.95	ms per image
clear, YOLOv8s	0.701	0.784	0.743	0.564	8.8
clear, RT-DETR-L	0.809	0.832	0.821	0.636	34.5
night, YOLOv8s	0.674	0.820	0.747	0.543	8.6
night, RT-DETR-L	0.733	0.787	0.760	0.580	34.7

Table 2: AP per class and mAP on each drive.

after 5 frames without a match. Table 3 gives the result at my confidence cut of 0.5, next to the truth row (the tracker run on the ground truth itself).

Drive, model (cut 0.5)	ID switches	MOTA	matched	missed	false tracks
clear, YOLOv8s	6	0.449	597	713	3
clear, RT-DETR-L	43	0.476	806	504	139
night, YOLOv8s	9	0.405	542	768	2
night, RT-DETR-L	28	0.467	732	578	92
truth (both drives)	*3*	*0.925*	*1215*	*95*	*0*

Table 3: Tracking against the actors' IDs, 1,310 ground-truth boxes per drive.

The truth row is the best this tracker can do with a perfect detector. It is not 1.0 because a track is only reported from its third matched frame: 38 of the 95 misses are the first two frames of each of the 19 times an object comes into view, 52 more are frames just after a track had to be restarted, and 5 are frames where the filtered box lagged behind the object (IoU below 0.5).

Why a cut of 0.5. I ran the tracker at 0.3, 0.5 and 0.7 (Table 4). At 0.3 RT-DETR-L's low-score boxes flood the tracker: 110 and 128 ID switches and MOTA down to 0.218 and 0.063. At 0.7 YOLOv8s keeps too few boxes and its MOTA falls to 0.250 and 0.164. 0.5 gives RT-DETR-L its best MOTA on both drives, keeps YOLOv8s within 0.05 of its best MOTA with 3 to 4 times fewer ID switches than at 0.3, and is the same threshold the node uses, so the tracker is graded on what the node publishes.

ID switches, MOTA	cut 0.3	cut 0.5	cut 0.7
clear, YOLOv8s	25, 0.470	6, 0.449	8, 0.250
clear, RT-DETR-L	110, 0.218	43, 0.476	8, 0.376
night, YOLOv8s	23, 0.457	9, 0.405	1, 0.164
night, RT-DETR-L	128, 0.063	28, 0.467	5, 0.367

Table 4: The choice of confidence cut.

6 The four questions (Task 5)

6.1 The budget

L4 gives each frame 50 ms at 20 Hz. On my GPU YOLOv8s uses 8.8 ms on the clear drive and 8.6 ms at night, 18% of the budget, which leaves about 41 ms. RT-DETR-L uses 34.5 ms and 34.7 ms, 69%, which leaves about 15 ms (Table 1). My tracker takes 0.05 ms per frame (median; 0.26 ms at the 99th percentile) on the CPU (Intel i9-12950HX), so tracking costs almost nothing. With YOLOv8s, planning has about 41 ms. With RT-DETR-L it has about 15 ms, and that has to cover the ROS messages and image conversion too, which `run_detector.py` does not time. Both models kept up with 20 Hz in the live node, but RT-DETR-L does so with little to spare.

6.2 Night

RT-DETR-L loses more AP at night. Its mAP@0.5 falls from 0.821 to 0.760 (−0.061) and its mAP@0.5:0.95 from 0.636 to 0.580 (−0.056). The larger AP loss is in the vehicle class (0.809 to 0.733, −0.076); pedestrians lose 0.045 (0.832 to 0.787). YOLOv8s hardly changes: mAP@0.5 goes from 0.743 to 0.747; its vehicle AP falls 0.027 and its pedestrian AP rises 0.036. Because both drives show the same scene in the same frame, I also counted objects one by one at the cut of 0.5. RT-DETR-L found 40 pedestrian boxes by day that it missed at night, and only 4 the other way round; for vehicles it is 97 against 54. YOLOv8s: 18 against 20 pedestrians. So in numbers of objects, RT-DETR-L's night problem is pedestrians.



(a) clear, RT-DETR-L: found, 0.82 **(b)** night, RT-DETR-L: missed (0.05) **(c)** night, YOLOv8s: found, 0.51

Figure 2: Frame 210, drawn with `show_frame.py -dets` (cut 0.5); crops of the left of the image. Blue: ground truth; red: detections with their scores. The pedestrian is the same in both drives (actor ID 311 by day, 577 at night).

Figure 2 shows one such miss. By day RT-DETR-L finds the pedestrian with a score of 0.82. At night its best box is still in the right place (IoU 0.88 with the ground truth) but its score falls to 0.05, so it is dropped at the cut. YOLOv8s finds the same pedestrian at night with 0.51. The pedestrian is about 30×60 px against a dark fence, lit only by the street light. Over the 182 frames this pedestrian is in view, RT-DETR-L finds them in 157 by day and 134 at night.

6.3 Tracking

RT-DETR-L gives my tracker more ID switches at the cut of 0.5: 43 against 6 on the clear drive and 28 against 9 at night. I looked at the two places the assignment suggests (Table 5).

Drive, model (cut 0.5)	same-class pairs, IoU > 0.5 (frames)	of them IoU > 0.95	gaps of 5+ frames	switches with old track alive	ID switches, MOTA after merging
clear, YOLOv8s	8 (8)	5	8	1 of 6	5, 0.450
clear, RT-DETR-L	76 (65)	75	13	39 of 43	5, 0.543
night, YOLOv8s	7 (7)	7	10	3 of 9	6, 0.409
night, RT-DETR-L	75 (68)	65	9	22 of 28	7, 0.504

Table 5: Where the ID switches come from. A gap is a run of frames in which an object in view has no box at the cut with IoU $\geq$ 0.5, between two frames where it has one. Merging: of two same-class boxes with IoU above 0.7, keep the one with the higher score.

Duplicate boxes are the cause. RT-DETR-L gives about ten times as many pairs of same-class boxes with IoU above 0.5: 76 on the clear drive against 8 for YOLOv8s, and 75 against 7 at night. Almost all of them (75 of 76) are the same box twice. When I ran RT-DETR-L on these frames and kept the raw COCO labels, the two copies had different labels: car and truck on the van in frame 66, bus and truck on the minibus in frame 88 (Figure 3). RT-DETR has no non-maximum suppression, so two of its queries

can land on the same object, and here they often give it different labels; GP2 maps car, truck and bus to *vehicle*, so the tracker sees two vehicle boxes on one van. The second box starts a second track, which is confirmed after 3 frames, and the two tracks then take turns on the same object. In 39 of the 43 switches on the clear drive (22 of 28 at night) the old track was still alive when the object moved to the new one. All 43 clear-drive switches are vehicles, and two vehicles (actors 214 and 221) cause 32 of them.



(a) RT-DETR-L: 2 boxes on the minibus, 3 on the van

(b) YOLOv8s: one box each

Figure 3: Clear frame 88 (cut 0.5). RT-DETR-L's two minibus boxes score 0.75 and 0.54 (bus and truck); the van's three score 0.62, 0.55 and 0.52 (truck, truck and car). Each set is the same box, so the scores are printed on top of each other.

Missed frames are not the cause. Gaps of 5 frames or more, long enough for a track to be deleted, are about as common for both models (13 against 8 by day, 9 against 10 at night). They explain most of YOLOv8s's few switches (in 5 of its 6 by day the old track had been deleted), but not RT-DETR-L's. As a check, I merged same-class boxes with IoU above 0.7 before the tracker, keeping the higher score. RT-DETR-L's ID switches fell from 43 to 5 and its MOTA rose from 0.476 to 0.543 on the clear drive; at night they fell from 28 to 7, MOTA from 0.467 to 0.504. YOLOv8s barely changed (6 to 5, 9 to 6). This merge is an offline test only; the graded numbers in Tables 3 and 4 are without it.

6.4 The recommendation

By day, RT-DETR-L, with duplicate vehicle boxes merged in `predict`. On the clear drive it is better by 0.078 mAP@0.5 (0.821 vs 0.743) and 0.072 mAP@0.5:0.95, with a higher AP for both vehicles (+0.108) and pedestrians (+0.048). With the merge its tracking is also better: 5 ID switches against 6 and MOTA 0.543 against 0.449. The cost is 25.7 ms more per frame: 69% of the budget instead of 18%, leaving about 15 ms instead of 41 ms for planning.

At night, YOLOv8s. At night RT-DETR-L's lead shrinks to 0.013 mAP@0.5 (0.760 vs 0.747), and it is worse on pedestrians, the class that matters most for safety: AP@0.5 0.787 against 0.820, and the 40 pedestrian boxes it loses at night. For that small gain it would still cost 4 times the time. Switching is a launch parameter (`model_type`), since both models sit behind the same interface.

If the team wants one model for both, I would pick YOLOv8s: it gives up 0.078 mAP@0.5 by day but only 0.013 at night, it is better on pedestrians at night, and it leaves 41 ms for the rest of the stack. Its weak point is MOTA: at the cut of 0.5 it misses more objects than RT-DETR-L (713 against 504 by day), so a lower cut for YOLOv8s alone is the next thing I would test.